

GitHub Component README Examples

Document field	Value
Document owner	FortVault
Intended audience	FortVault's ISO 27001 implementation partner
Classification	Confidential
Status	Sanitized source-controlled examples
Date	August 28, 2026

Purpose and Handling

This document provides representative examples from selected FortVault GitHub component READMEs. The excerpts demonstrate repository documentation practices for component purpose, architecture boundaries, secure configuration, testing, health checks, and operational setup.

The export is intentionally sanitized. It omits partner-specific information, local absolute paths, credentials, private endpoints, generated MPC configuration, secret values, and recovery material. It is evidence of source-controlled documentation, not proof that a production control is configured or effective.

Reviewed README Revisions

Component	Repository README	Branch	Reviewed commit
Backend	fortvault-backend/README.md	development	45b6cf8082a
Custody Processing	fortvault-processing/README.md	development	e9c563ba8b41
MPC	fortvault-mpc/README.md	development	c01f0f6897c8
Health Audit	fortvault-health-audit/README.md	main	3193b4619ec3

1. Backend README Example

Component description

The Backend README identifies the service as a NestJS application using Sign-In with Ethereum authentication, JSON Web Tokens, PostgreSQL with TypeORM, Redis, Docker, and generated API documentation.

Authorization documentation

The README documents role-based access control through canonical colon-separated permission identifiers. Controllers reference the same canonical permission tree that is used when roles and permissions are seeded, helping keep route authorization and stored permission values aligned.

Secure setup guidance

- Private package registry credentials are supplied through environment or CI secret configuration rather than committed values.

- Two-factor authentication secrets are encrypted with an externally supplied 256-bit encryption key.
- Database migrations are documented as a required deployment step.
- Production configuration supports AWS Secrets Manager and PostgreSQL SSL settings.
- Database schema synchronization is documented as disabled for deployed environments.

Declared quality commands

- Unit tests: `npm run test`
- End-to-end tests: `npm run test:e2e`
- Test coverage: `npm run test:cov`
- Lint: `npm run lint`
- Build: `npm run build`

2. Custody Processing README Example

Component description and ownership

The Custody Processing README identifies this service as the custody execution orchestrator. It is the component authorized to coordinate MPC key/sign operations and broadcast blockchain transactions. Each stateful service owns its own database, while Redis Streams is documented as transport rather than durable business state.

Documented execution behavior

- Address generation uses pre-generated partner root pools and reserves derivation indexes before contacting MPC.
- Transfer execution has explicit progressing and failure states.
- Retryable work uses exponential backoff and transitions to a final failure after the configured maximum attempts.
- Inbound messages are idempotent by message identifier and reject payload mismatches.
- Durable inbox, outbox, request, root, address, MPC, and transfer state is stored in Processing PostgreSQL.

Secure deployment guidance

The README separates required database, Redis, broker, policy, blockchain RPC, and MPC configuration. It documents AWS Secrets Manager loading, PostgreSQL SSL settings, strict chain-RPC configuration, and non-production-only test endpoints.

Declared quality commands

- Unit tests: `npm test`
- End-to-end tests: `npm run test:e2e`
- Lint: `npm run lint`
- Build: `npm run build`

3. MPC README Example

Component description

The MPC README describes distributed key generation and signing, node configuration generation, extraction of encrypted key parts, and recovery tooling. It identifies the MPC node as the cryptographic boundary and keeps key shares and MPC session state in node-owned storage.

Configuration-generation safety

The documented configuration generator:

It validates the node host count, creates the complete result in a private temporary sibling directory, and refuses to overwrite an existing destination. Node configuration contains separate P2P identity, database, attestation, and key-share encryption material. Real service API credentials must be injected from the deployment secret manager rather than generated or committed.

Trustless deployment guidance

For independently operated nodes, the README recommends that each participant generate its own local identity and private configuration, then exchange only public identity keys, node identifiers, and routing endpoints. This prevents a central generator from learning every participant's private material.

Declared quality command

- Go test suite: `go test ./...`

4. Health Audit README Example

Component description

The Health Audit README describes a read-only client for standardized service and MPC health endpoints. It checks liveness, readiness, sanitized configuration, expected partner/policy context, and Redis stream/consumer-group consistency.

Secret and manifest handling

The audit key is supplied at runtime through an environment variable or secret manager. Environment manifests contain service URLs and expected configuration but no secret values. Separate key references can be configured per service or MPC cluster. Text output supports operators, while stable JSON output supports CI and automation.

Declared verification and exit behavior

Automated tests use `npm test`. Exit code 0 means healthy, 1 means failed health or configuration consistency, and 2 means invalid audit setup.

Evidence Limitations

- These examples show documentation maintained in source control; they do not prove that every production deployment follows the documented process.
- Release-specific test output, deployment approvals, cloud configuration, backup evidence, centralized logging, TLS settings, and endpoint compliance require separate operational evidence.
- Repository access should remain restricted. This reviewed export is preferable to granting broad GitHub access solely for the ISO evidence request.